UTEC
Universidad Tecnológica

Tests unitarios

Pruebas Unitarias en Java: ¿Qué son y para qué sirven?

¿Qué son las pruebas unitarias?

Las **pruebas unitarias** son un tipo de prueba de software que verifica el comportamiento de la unidad más pequeña del código, como una función o método, de manera aislada. El propósito de una prueba unitaria es asegurar que cada parte individual del programa funcione correctamente por sí misma, sin depender de otras partes del sistema.

Las pruebas unitarias:

- **Verifican la funcionalidad**: Aseguran que los métodos o funciones de una clase produzcan los resultados esperados con datos específicos.
- **Detectan errores temprano**: Al probar cada unidad individual, es más fácil identificar y corregir errores en fases tempranas del desarrollo.
- **Facilitan el mantenimiento**: Al agregar nuevas funcionalidades o realizar modificaciones, las pruebas unitarias permiten asegurarse de que no se rompa el comportamiento existente.

Ventajas de las pruebas unitarias

1. **Mejoran la calidad del código**: Al verificar cada unidad de código, los desarrolladores pueden detectar errores rápidamente.
2. **Facilitan la refactorización**: Si el código se cambia o se mejora, las pruebas aseguran que el comportamiento previo no se vea afectado.
3. **Documentación viviente**: Las pruebas actúan como una documentación para entender el comportamiento esperado de las funciones y métodos.
4. **Desarrollo ágil**: Son fundamentales para metodologías de desarrollo como TDD (Test-Driven Development), donde se escribe el código de prueba antes de implementar la funcionalidad.

Herramientas y librerías para pruebas unitarias en Java

Las pruebas unitarias en Java se pueden realizar utilizando varias librerías y frameworks, siendo los más comunes:

- **JUnit**: Es el framework de pruebas unitarias más utilizado en Java. JUnit permite escribir y ejecutar pruebas de manera estructurada. Su última versión es JUnit 5.

- **Mockito**: Esta biblioteca se usa en combinación con JUnit para realizar **mocking** (creación de objetos falsos que simulan el comportamiento de objetos reales), lo que permite aislar las unidades de prueba.
- **AssertJ**: Proporciona un conjunto de aserciones más expresivo y legible que el que ofrece JUnit de forma nativa.
- **Hamcrest**: Biblioteca que permite escribir reglas de aserción más complejas para pruebas unitarias.

Ejemplo Simple de Prueba Unitaria: Calculadora con JUnit

Veamos un ejemplo simple de cómo realizar pruebas unitarias usando **JUnit**. En este ejemplo, probaremos una clase `Calculadora` que tiene métodos para sumar, restar, multiplicar y dividir.

1. Crear la clase `Calculadora`

```
public class Calculadora {

    public int sumar(int a, int b) {
        return a + b;
    }

    public int restar(int a, int b) {
        return a - b;
    }

    public int multiplicar(int a, int b) {
        return a * b;
    }

    public int dividir(int a, int b) throws IllegalArgumentException {
        if (b == 0) {
            throw new IllegalArgumentException("No se puede dividir por
cero");
        }
        return a / b;
    }
}
```

2. Crear las pruebas unitarias usando JUnit

Para las pruebas, usaremos **JUnit 5**. Debemos agregar la dependencia de JUnit al archivo `pom.xml` de Maven:

```
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-api</artifactId>
    <version>5.8.1</version>
    <scope>test</scope>
</dependency>
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-engine</artifactId>
    <version>5.8.1</version>
    <scope>test</scope>
</dependency>
```

3. Escribir las pruebas en JUnit

```
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;

class CalculadoraTest {

    private final Calculadora calculadora = new Calculadora();

    @Test
    void testSumar() {
        assertEquals(5, calculadora.sumar(2, 3), "2 + 3 debería ser 5");
    }

    @Test
    void testRestar() {
        assertEquals(1, calculadora.restar(3, 2), "3 - 2 debería ser
1");
    }

    @Test
    void testMultiplicar() {
        assertEquals(6, calculadora.multiplicar(2, 3), "2 * 3 debería
```

UTEC
Universidad Tecnológica

```
ser 6");
    }

    @Test
    void testDividir() {
        assertEquals(2, calculadora.dividir(6, 3), "6 / 3 debería ser
2");
    }

    @Test
    void testDividirPorCero() {
        Exception exception =
assertThrows(IllegalArgumentException.class, () ->
calculadora.dividir(6, 0));
        assertEquals("No se puede dividir por cero",
exception.getMessage());
    }
}
```

4. Explicación del código de prueba

- `assertEquals(expected, actual)`: Compara el valor esperado con el valor actual devuelto por el método probado. Si no coinciden, la prueba falla.
- `assertThrows()`: Verifica que se lance una excepción cuando se intenta realizar una operación no válida, en este caso dividir por cero.

5. Ejecutar las pruebas en IntelliJ

1. Haz clic derecho sobre la clase de pruebas `CalculadoraTest` y selecciona **Run 'CalculadoraTest'**.
2. IntelliJ ejecutará las pruebas y mostrará los resultados en la ventana de la consola. Si todas las pruebas pasan, aparecerá un mensaje indicando el éxito de las mismas.